

VIDEO LINK: https://mediaspace.msu.edu/media/group-06-classes/1_rk9gzskl

MemoryFactory Script

Hi everyone, my name is Joshua Thomas, and today I will be talking about the MemoryFactory class.

MemoryFactory is a fixed-size pool allocator for objects of type T, and it uses a freelist to make allocation and reuse really fast. The main idea is that instead of calling new and delete every time you create or destroy an object, the allocator grabs memory in bigger chunks called blocks. Each block is split into a fixed number of slots, and each slot can hold exactly one object.

Having large contiguous blocks also helps with cache locality.

Those slots are tracked by a FreeList, which is just a singly-linked list of available slots. When I need memory, I pop a slot from the freelist in $O(1)$ time, and when I am done with an object, I push that same slot right back, also in $O(1)$ time.

When I call Make(...), the factory takes one of those free slots and constructs an object in place using placement new. If the freelist ever runs out of slots, it just allocates a new block and adds all of its slots to the freelist. When I'm done with an object, I call Delete(ptr), which explicitly runs the destructor and returns the slot back to the freelist for reuse.

For safer usage, I also provide a MakeUnique(...) function, which returns a `std::unique_ptr` with a custom deleter. That means when the smart pointer goes out of scope, it automatically returns the object to the pool. The only rule is that the pool has to outlive any of these smart pointers, since the deleter stores a pointer back to the pool.

In the demo, I first show the raw Make and Delete workflow. I allocate a couple of Widget objects, delete them, and then allocate again. The addresses often repeat, which shows that the same slots are being reused instead of going back to the system allocator. Then I show MakeUnique, where the objects are automatically destroyed and returned to the pool when they go out of scope.

BehaviorTree Script

To handle the complexity of classic agents, we implemented a custom Behavior tree engine based on data and logic. It starts with the Blackboard; a "shared brain" of our agent. Using a C++ `std::map` and `std::variant`, we created a flexible storage system. This allows us to store anything from integers like "Gold count" to booleans like "Is Under attack" under a single interface. By passing Blackboard by reference throughout the tree, every node stays synchronized.

The Body of our agent is a hierarchical tree. When the game engine 'ticks,' it signals the Root Node to begin. The signals flow through Composite Nodes, like our Sequences and Selectors. The nodes have different statuses like Success (1), Failure (0), or Running (-1). A Sequence acts

like a strict 'To-Do list,' only moving forward if every child succeeds. A Selector, on the other hand, acts like a 'Decision Maker,' trying different strategies until one finally works.

The real work happens at the Leaf Nodes. Instead of hardcoding every behavior into a new class, we use `std::function` wrappers. When we create a node, we pass in a lambda as a parameter. Condition Nodes act as 'Sensors' that return true or false based on the Blackboard, while Action Nodes act as the 'Hands' that physically modify the Blackboard values.

The decorator nodes modify the result of a single child. An inverter will invert the status of a node. For example, If its status is success, it'll flip to failure.

Finally, to ensure our engine works perfectly for a group project environment, we used Assertions, as well as catch2 test cases. This helps ensure the class can handle many outlier and expected behavior.

WorldPath Script

Hello, I am Bryant Shepherd and this is my World Path Class. The purpose of this class is to represent a 2D path as an ordered sequence of points. These points are stored using a point struct with x and y coordinates represented as doubles. Privately, this class stores a vector of these points called mPoints. This is important for this class because the vector is able to be dynamically resized as more points are added to the path as well as preserving insertion order. This class contains simple path functions such as add, clear, reverse and start and end to return the start and end points respectively. The length function iterates through the points and sums the distance between them to return the total length of the path, this uses the Dist helper function to find the distance between two points. Lastly, the self_intersect function detects whether or not the path has intersected itself previously in the world, with the help of a few helper functions. First, Cross computes a 2D cross product, which gives us a signed area which is useful to know the orientation. Next, OnSegment checks if a point p lies on a segment of AB. SignWithEps is for preventing floating point precision errors by checking if a value is greater, less than or equal to a value. SegmentsIntersect is the main intersection test between segments AB and CD. It computes four cross products to tell us which side of a segment the point is on, then convert them into signs using SignwithEps. This main intersection condition checks if the points C and D are on opposite sides of AB as well as if A and B are on opposite sides of CD, If so they intersect. These other handlers are for if any cross product is 0 and the point lies on a segment, they intersect. If none of those conditions are met, the segments do not intersect. So with the main SegmentsIntersect test, self_intersect checks whether there is any intersection throughout the whole path.

PathGenerator Script

Hi, my name is Matthew Vazquez and this is the Path Generator class. This class was made in order to have agents be able to navigate through world objects. This class interacts with a world view class, which has the functionality of seeing the position neighbors and checking if a position is walkable. The path generator class also returns a world path object, which is the path object that is generated. The path generator interacts with world view through function calls of get neighbors and is walkable. The world view interacts with path generator by calling path requests into the path generator class. These path requests have a path type, start, goal, max length, and position to avoid. The path request is then funneled into the path type that is selected. This is the sample shortest path generator, which generates the shortest path based on the start and goal. It uses a breadth-first search, ensuring the shortest path is found, if there is a suitable one. It will then either return the world path object, which will either be empty when no path is found, or will have the desired path mapped out from start to end.

ActionMap Script:

The ActionMap class creates a system for mapping string names to executable void functions. This allows us to trigger code dynamically based on text input. This is especially useful for connecting UI buttons to game logic or building a console where commands need to be looked up by name. Instead of hardcoding every connection, we can register new behaviors at runtime using AddFunction, or swap them out using ReplaceFunction. We used an std::any to store the functions generically and std::type_index to ensure we don't accidentally try to run data that isn't a valid function.

The core of the class is the Trigger function, which handles the actual execution safely. Before running anything, it verifies that the command exists and that the stored type is correct. It also wraps the execution in a try-catch block, so if the stored function throws an exception or fails to cast, the engine won't crash. Instead, it returns an error message describing the problem. We also included utilities to remove actions and retrieve a sorted list of all registered names, which makes it easy to generate a list of available commands for the user interface.